



Программирование на Python

Списки, словари, кортежи, множества

План лекции

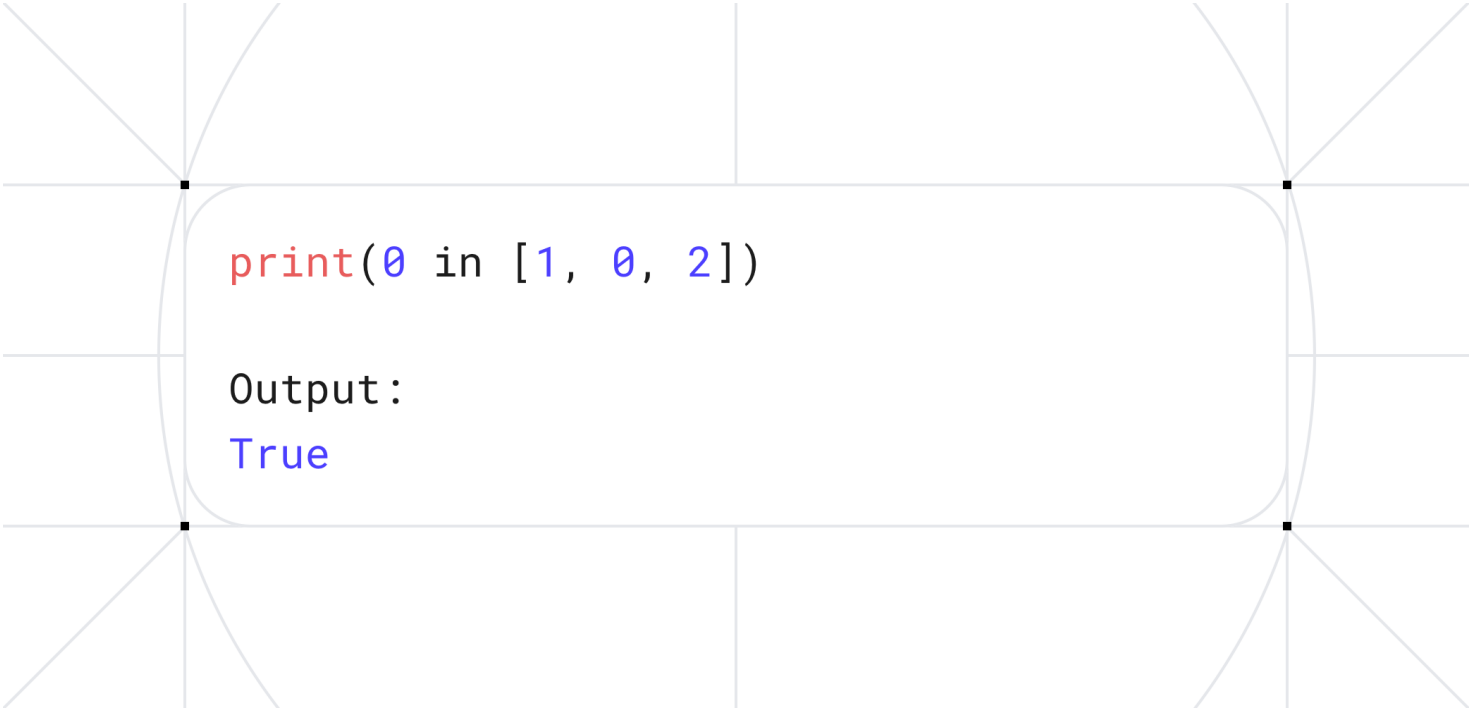
- Коллекции
- Списки
- Кортежи
- Множества
- Словари

Коллекции в Python

- [list](#) – списки;
- [tuple](#) – кортежи;
- [range](#) – диапазоны;
- [str](#) – строки;
- [bytearray](#) и [bytes](#) – изменяемый и неизменяемый массивы байтов;
- [set](#) и [frozenset](#) – изменяемые и неизменяемые множества;
- [dict](#) – словари (ассоциативные массивы, хеш таблицы).

Контейнер. Container

- `x` (does it `contain` A?)
- `x in A`



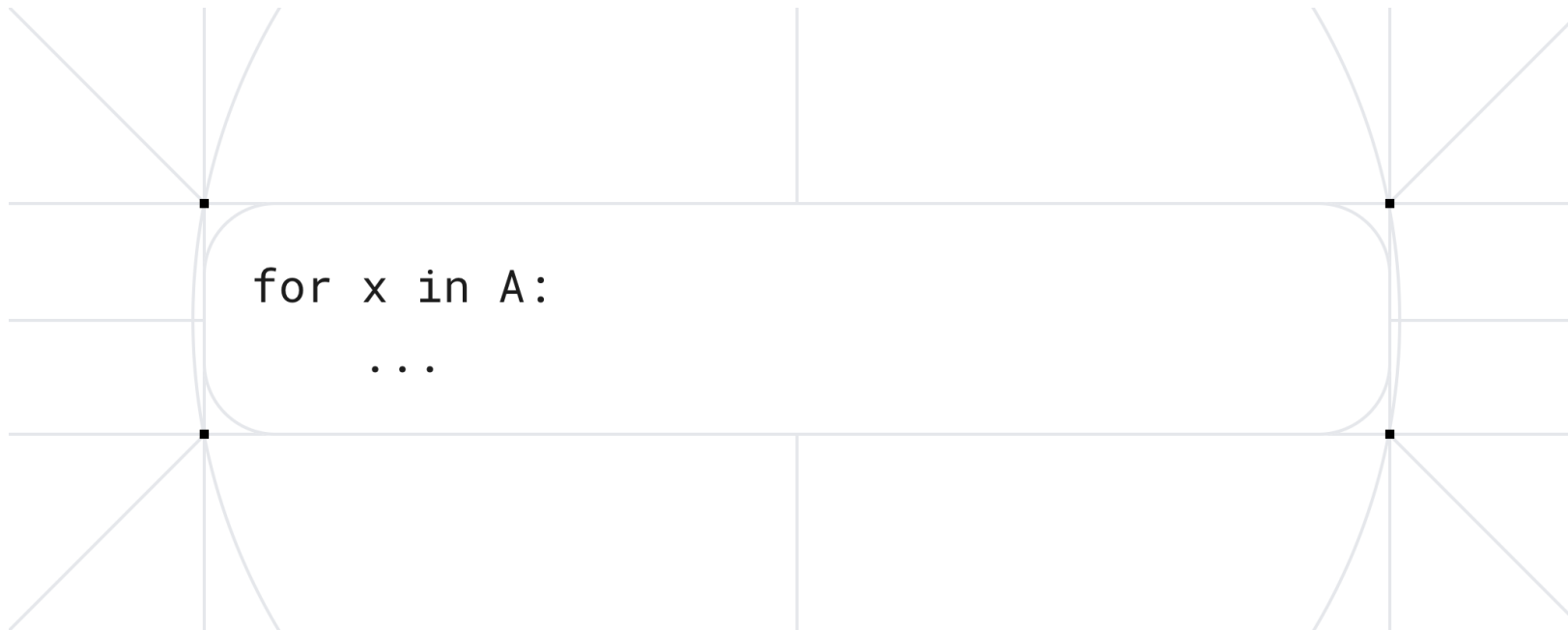
```
print(0 in [1, 0, 2])
```

Output:

True

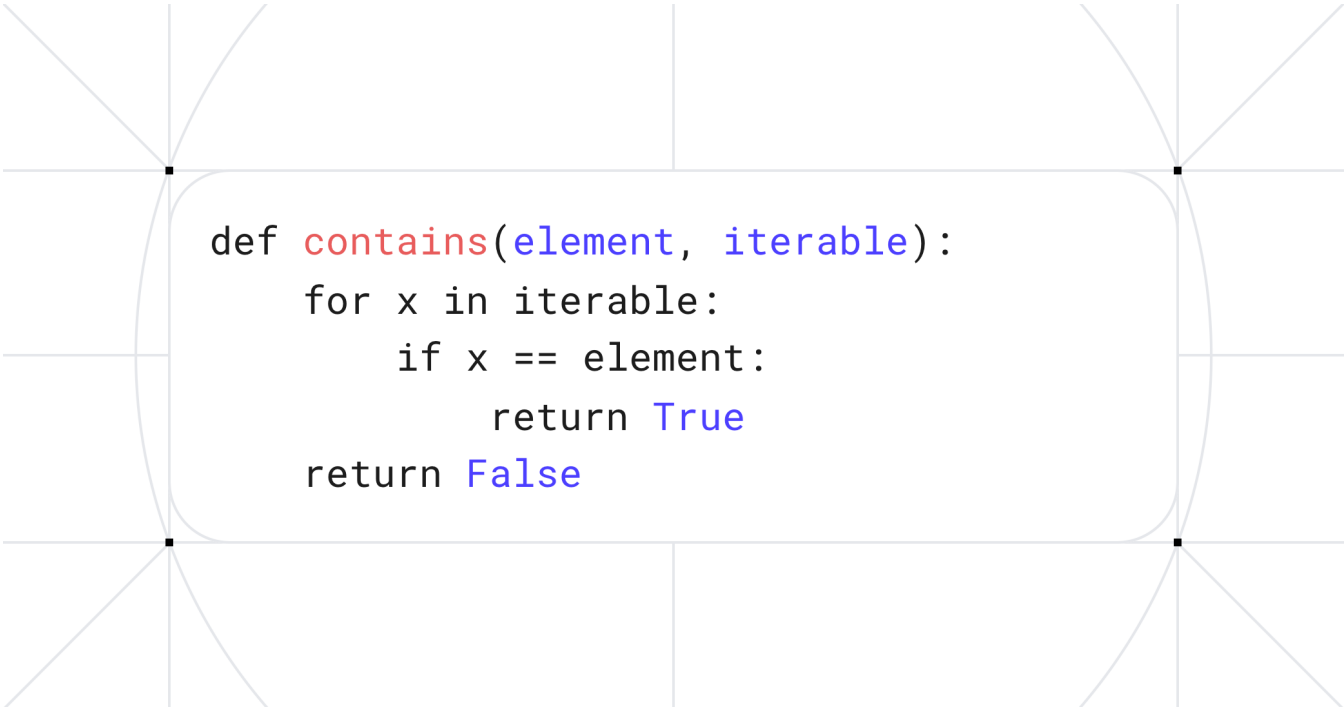
Итерируемый объект. `Iterable`

- можно пройти в цикле (`iterate over`)
- `x in A`



Итерируемый объект. `Iterable`

- `element in iterable`



```
def contains(element, iterable):  
    for x in iterable:  
        if x == element:  
            return True  
    return False
```

Объект ограниченной длины. Sized

- len(A)

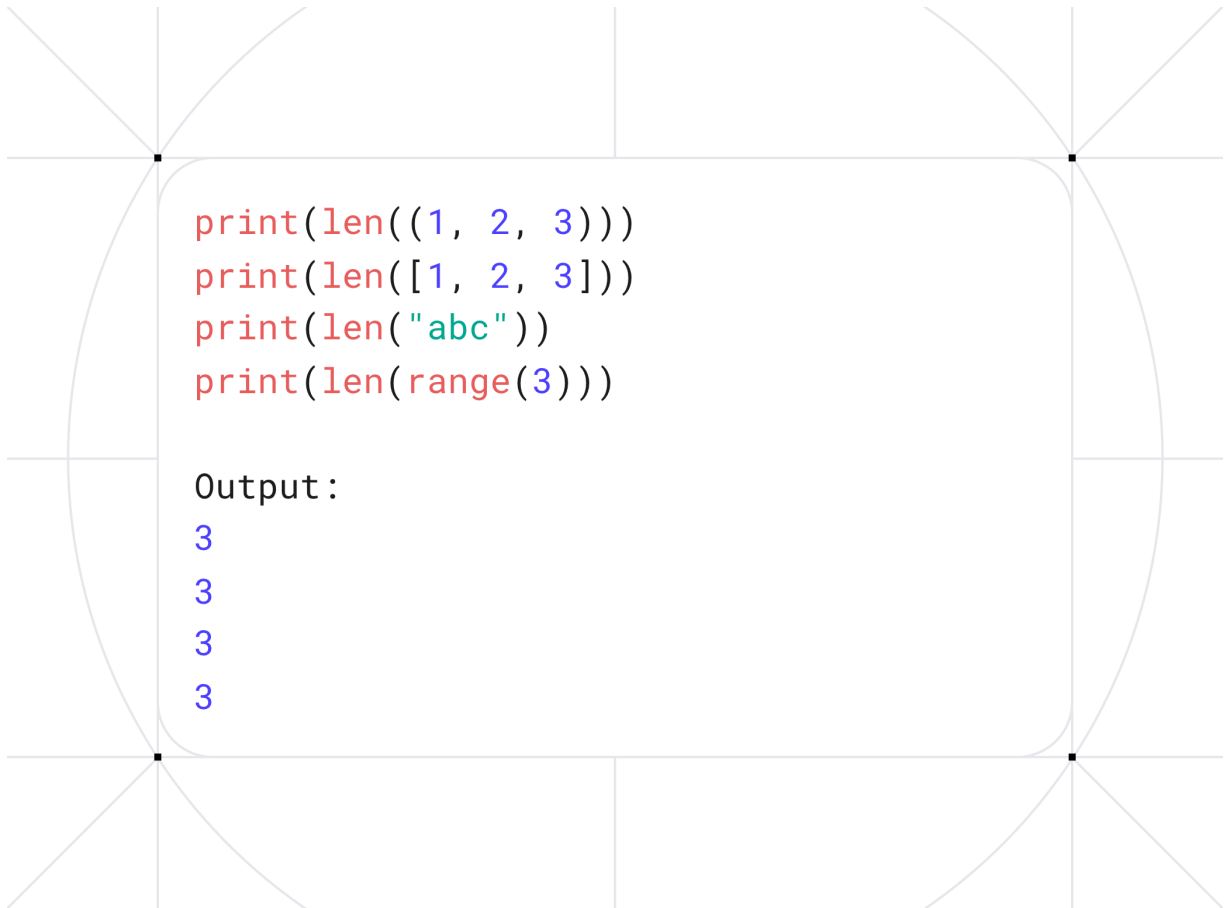
```
print(len((1, 2, 3)))  
print(len([1, 2, 3]))  
print(len("abc"))  
print(len(range(3)))
```

Output:

```
3  
3  
3  
3
```

Объект ограниченной длины. Sized

- len(A)



```
print(len((1, 2, 3)))  
print(len([1, 2, 3]))  
print(len("abc"))  
print(len(range(3)))
```

Output:

```
3  
3  
3  
3
```

Контейнер, но не итерируемый и без длины

- $(a, b) = \{x \in \mathbb{R} \mid a < x < b\}$

```
from collections.abc import Container
from dataclasses import dataclass

@dataclass
class Interval(Container):
    a: float
    b: float

    def __contains__(self, x: float) -> bool:
        return self.a < x < self.b

interval = Interval(0, 1)
print(interval)

Output:
Interval(a=0, b=1)
```

Контейнер, но не итерируемый и без длины

```
numbers = (-0.5, 0.5, 1.5)
```

```
for x in numbers:  
    contains = x in interval  
    print(f"{x=}: {contains=}")
```

Output:

```
x=-0.5: contains=False
```

```
x=0.5: contains=True
```

```
x=1.5: contains=False
```

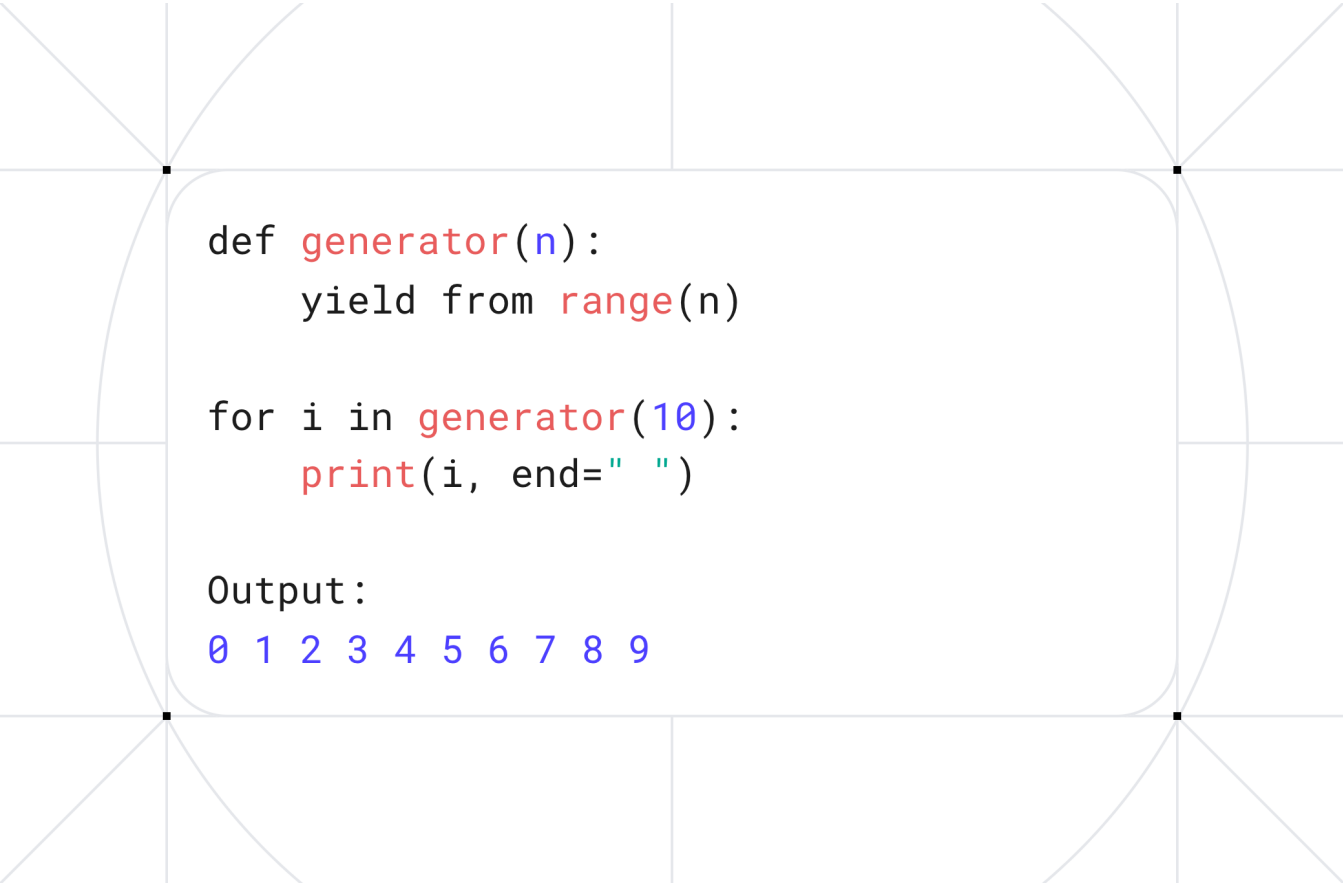
Контейнер, но не итерируемый и без длины

```
from collections.abc import Container, Iterable, Sized  
  
print(isinstance(interval, Container))  
print(isinstance(interval, Iterable))  
print(isinstance(interval, Sized))
```

Output:

```
True  
False  
False
```

Итерируемый, но не контейнер и безграничный



```
def generator(n):  
    yield from range(n)  
  
for i in generator(10):  
    print(i, end=" ")
```

Output:

0 1 2 3 4 5 6 7 8 9

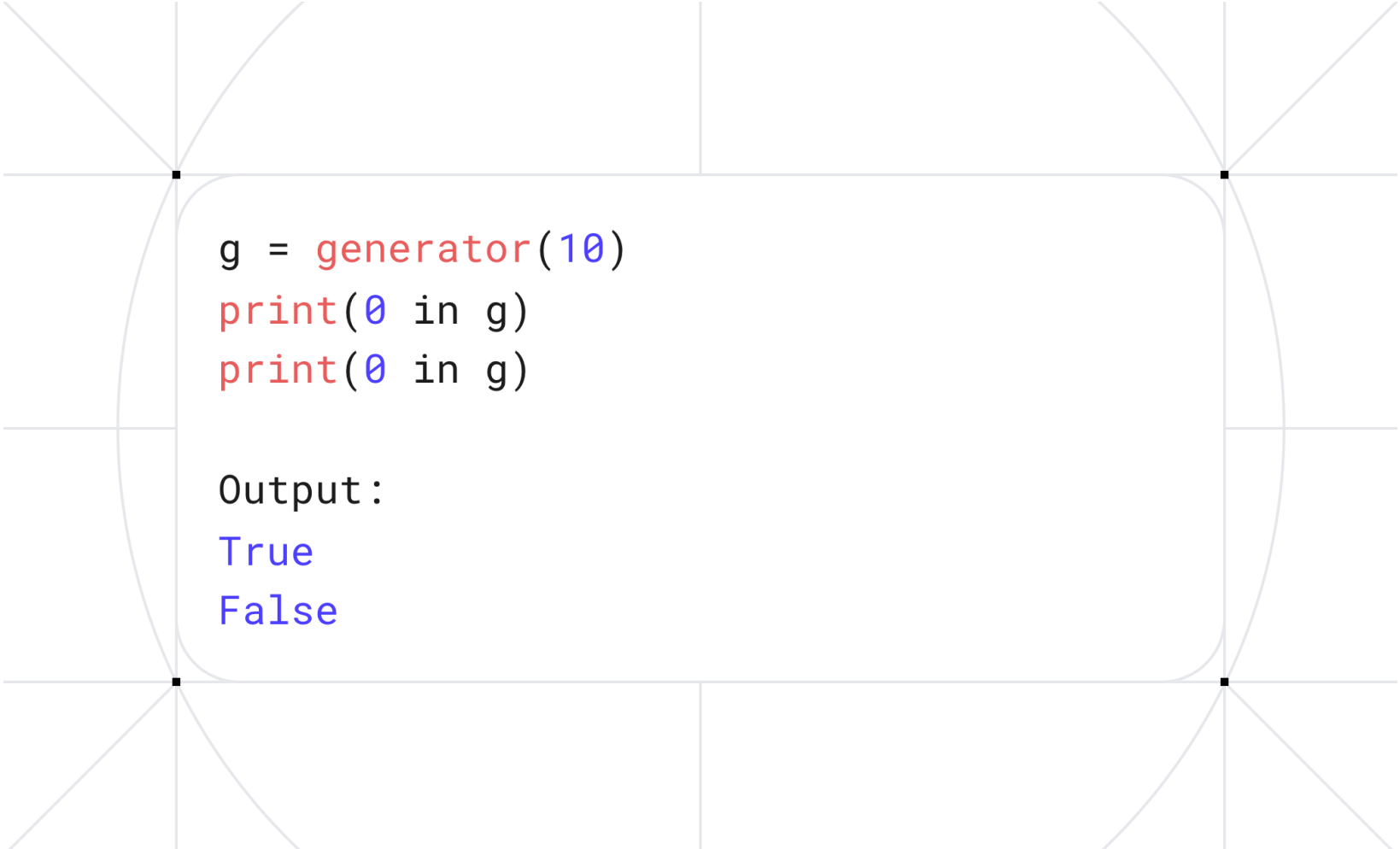
Итерируемый, но не контейнер и безграничный

```
len(generator(10))
```

Output:

```
-----  
TypeError                                Traceback (most recent call last)  
Cell In[7], line 1  
----> 1 len(generator(10))  
  
TypeError: object of type 'generator' has no len()
```

Итерируемый, но не контейнер и безграничный



```
g = generator(10)
print(0 in g)
print(0 in g)
```

Output:

```
True
False
```

Итерируемый, но не контейнер и безграничный

```
from collections.abc import Container, Iterable, Sized
```

```
print(isinstance(g, Container))
```

```
print(isinstance(g, Iterable))
```

```
print(isinstance(g, Sized))
```

Output:

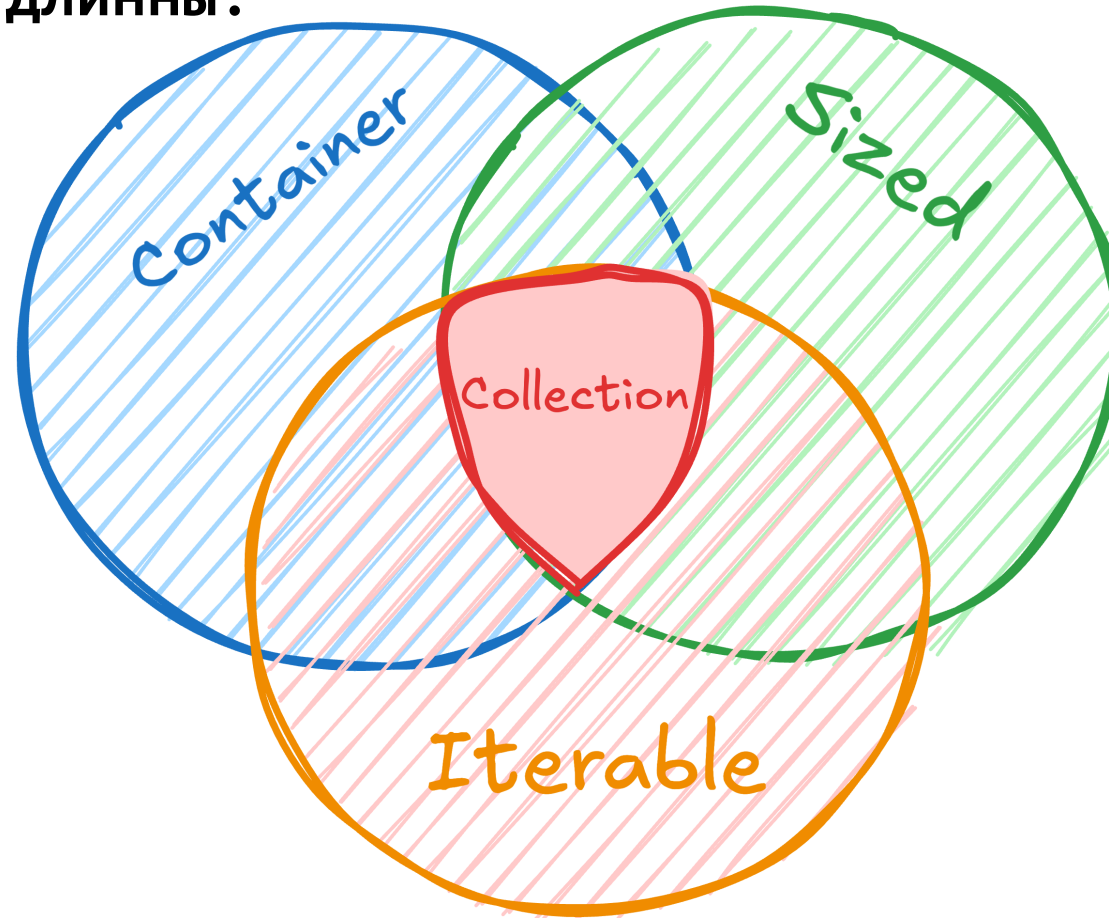
```
False
```

```
True
```

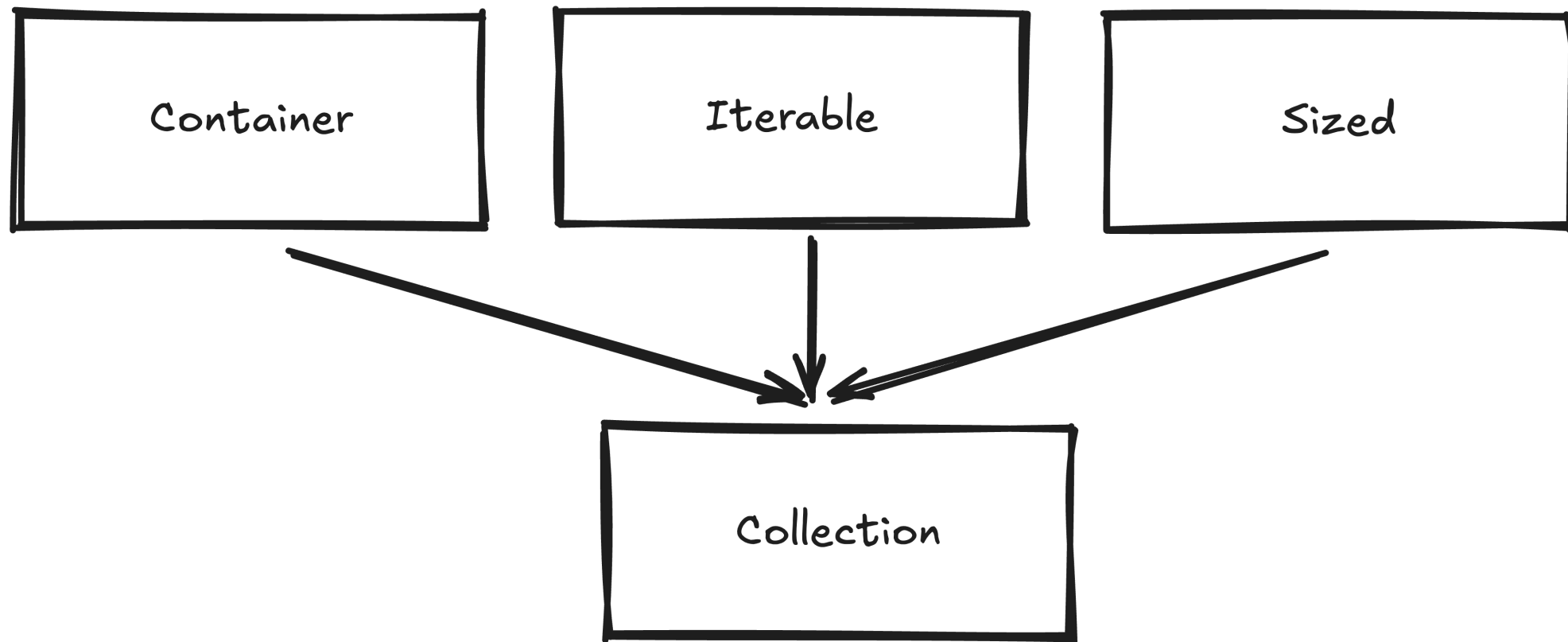
```
False
```

Коллекции

- **Коллекция** — объект, который одновременно является контейнером, итерируемым объектом и объектом ограниченной длины.



Коллекции



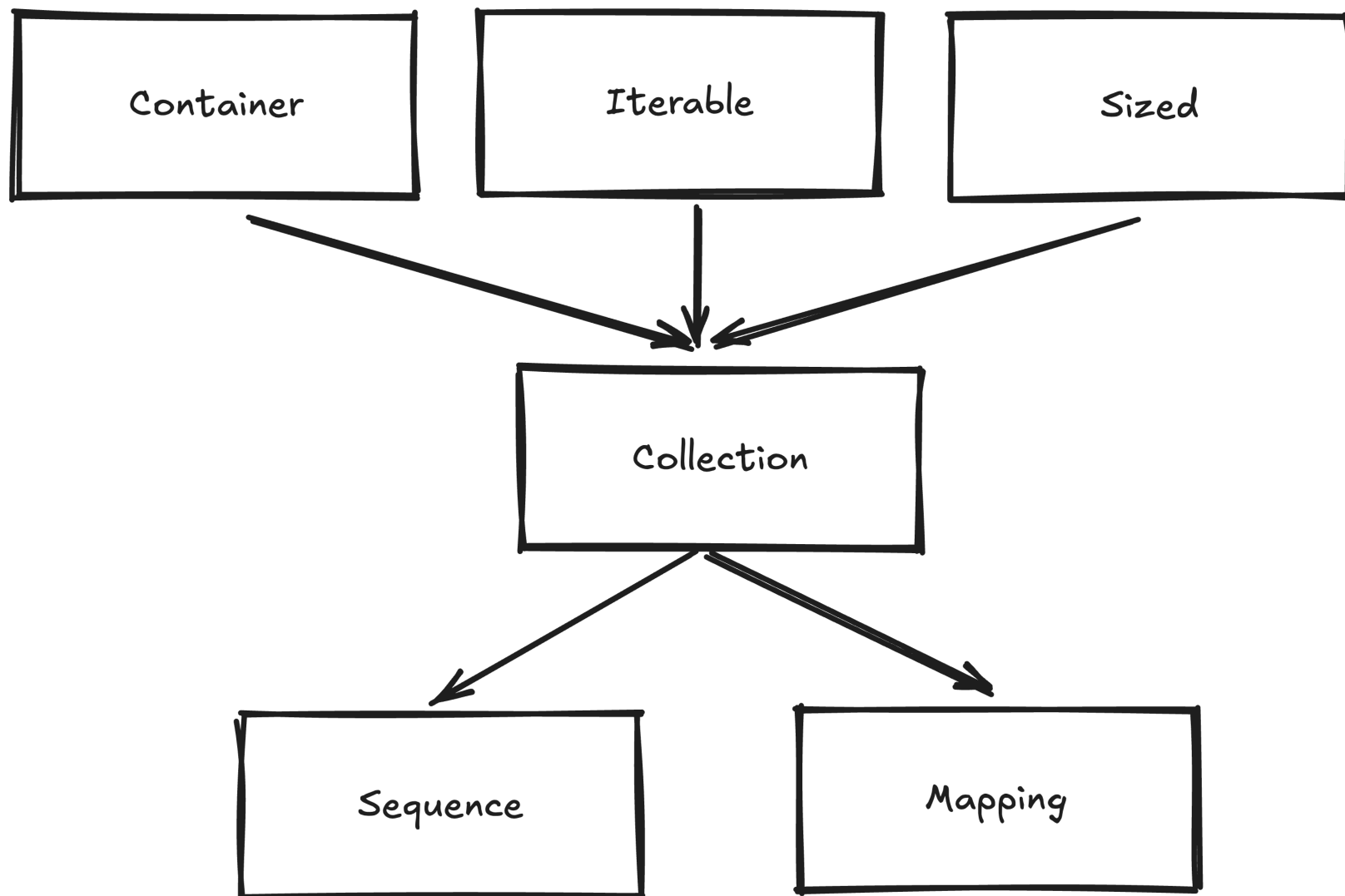
Коллекции

```
from collections.abc import Collection, Container, Iterable, Sized  
  
print(issubclass(Collection, Container))  
print(issubclass(Collection, Iterable))  
print(issubclass(Collection, Sized))
```

Output:

```
True  
True  
True
```

Коллекции



Коллекции

```
from collections.abc import Collection, Sequence, Mapping  
  
print(issubclass(Sequence, Collection))  
print(issubclass(Mapping, Collection))
```

Output:

True

True

Последовательности

- [list](#) – списки;
- [tuple](#) – кортежи;
- [range](#) – диапазоны;
- [str](#) – строки;
- [bytearray](#) и [bytes](#) – изменяемый и неизменяемый массивы байтов.

Последовательности

Операция	Описание
<code>s[i]</code>	<code>i</code> -й элемент последовательности <code>s</code> .
<code>s[i:j]</code>	Срез <code>s</code> от <code>i</code> включительно до <code>j</code> не включительно.
<code>s[i:j:k]</code>	Срез <code>s</code> от <code>i</code> включительно до <code>j</code> не включительно с шагом <code>k</code> .

Общие для последовательностей операции

Операция	Описание
<code>len(s)</code>	Длина <code>s</code> .
<code>x in s</code>	True, если в <code>s</code> есть элемент равный <code>x</code> . False иначе.
<code>x not in s</code>	Отрицание <code>x in s</code> .
<code>s + t</code>	Конкатенация <code>s</code> и <code>t</code> . Новый объект.
<code>s * n</code> или <code>n * s</code>	Эквивалентно добавлению <code>s</code> к самому себе <code>n</code> раз. Новый объект.
<code>min(s)</code>	Наименьший элемент в <code>s</code> .
<code>max(s)</code>	Наибольший элемент в <code>s</code> .
<code>s.index(x[, start[, end]])</code>	Индекс первого совпадения <code>x</code> с элементом в <code>s</code> . Бросает ошибку, если элемента нет в <code>x</code> . (В срезе <code>s[start:end]</code> , если метод вызван с <code>start</code> и/или <code>end</code> , но возвращаемое значение все равно отсчитывается от начала <code>s</code> , а не от <code>start</code>).
<code>s.count(x)</code>	Количество раз <code>x</code> встречается в <code>s</code> .

Принадлежность контейнеру

```
s = "abc"  
print("ab" in s)  
print("ac" in s)
```

Output:

```
True  
False
```

```
s = "Hello, World!"  
i = s.index("o")  
print(i, s[i])
```

Output:

```
4 o
```

Упорядочивание элементов последовательности

```
print("A" < "Z")  
print("A" < "a")
```

Output:
True
True

Упорядочивание элементов последовательности

```
x = "python"  
print(min(x), max(x))
```

Output:
h y

Упорядочивание элементов последовательности

```
x = "python"  
print(sorted(x))  
print(sorted(x, reverse=True))
```

Output:

```
['h', 'n', 'o', 'p', 't', 'y']  
['y', 't', 'p', 'o', 'n', 'h']
```

Упорядочивание элементов последовательности

```
chicken = chr(0x1f414)
egg = chr(0x1f95a)
print(chicken, egg)
print(sorted([egg, chicken]))
```

Output:



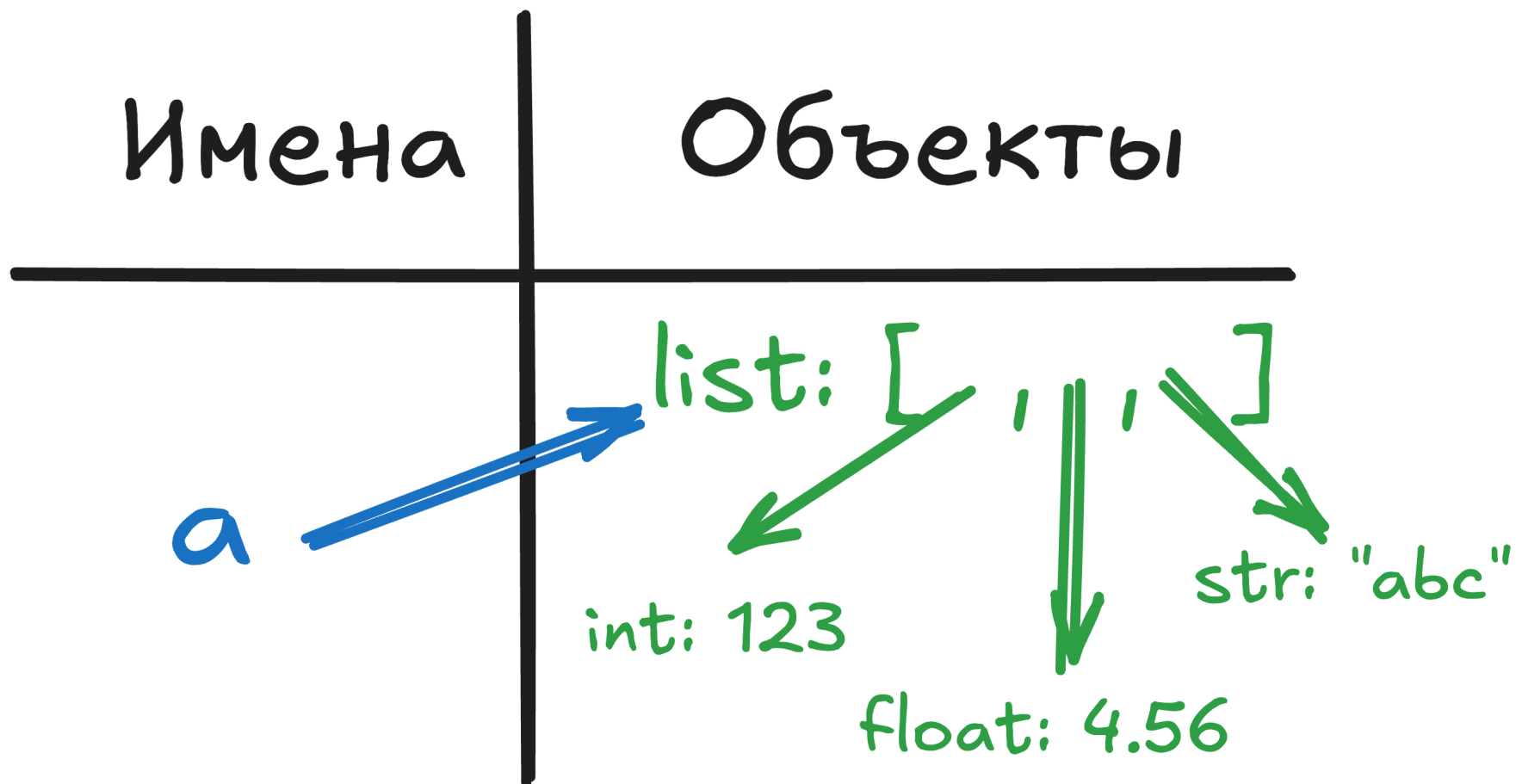
['🐔', '🥚']

Списки. `list`

- **Упорядоченная, изменяемая (`mutable`)** коллекция объектов произвольного типа.
- **Ключевые свойства:**
 - **Упорядоченный (`Ordered`):** Порядок элементов зафиксирован. Элемент, который вы добавили первым, будет иметь индекс 0.
 - **Изменяемый (`Mutable`):** Список можно изменять после его создания: добавлять, удалять, заменять элементы.
 - **Динамический (`Dynamic`):** Размер списка не фиксирован и может расти или уменьшаться по мере необходимости.
 - **Гетерогенный (`Heterogeneous`):** Может хранить объекты абсолютно разных типов данных одновременно.

Списки. `list`

- `a = [12, 4.56, "abc"]`



Создание списка

- литерал `[]`

```
empty_list = []  
numbers = [2, 3, 5, 7, 11]  
print(empty_list, type(empty_list))  
print(numbers, type(numbers))
```

Output:

```
[] <class 'list'>  
[2, 3, 5, 7, 11] <class 'list'>
```

Создание списка

- конструктор `list()`

```
empty_list = list()

tuple_example = (1, 2, 3)
list_from_tuple = list(tuple_example)

string_example = "NSU"
list_from_string = list(string_example)

print(empty_list, type(empty_list))
print(list_from_tuple, type(list_from_tuple))
print(list_from_string, type(list_from_string))

Output:
[] <class 'list'>
[1, 2, 3] <class 'list'>
['N', 'S', 'U'] <class 'list'>
```

Индексация списка

```
letters = ['a', 'b', 'c', 'd', 'e']  
print(letters[0])  
print(letters[-1])  
print(letters[6])
```

Output:

a
e

```
-----  
IndexError                                Traceback (most recent call last)  
Cell In[20], line 4  
      2 print(letters[0])  
      3 print(letters[-1])  
----> 4 print(letters[6])
```

```
IndexError: list index out of range
```

Срезы

- `my_list[start:stop:step]`

```
numbers = [0, 1, 2, 3, 4, 5, 6]
```

```
print(numbers[1:4])
```

```
print(numbers[:3])
```

```
print(numbers[3:])
```

Output:

```
[1, 2, 3]
```

```
[0, 1, 2]
```

```
[3, 4, 5, 6]
```

Срезы

```
numbers = [0, 1, 2, 3, 4, 5, 6, 7, 8]
```

```
print(numbers[::2])
```

```
print(numbers[1:8:3])
```

```
print(numbers[::-1])
```

Output:

```
[0, 2, 4, 6, 8]
```

```
[1, 4, 7]
```

```
[8, 7, 6, 5, 4, 3, 2, 1, 0]
```

Добавление элементов

- **`.append(item)` – $O(1)$**
 - Добавляет **один** элемент в конец списка.
 - `nums = [1, 2]; nums.append(3) # -> [1, 2, 3]`
- **`.extend(iterable)` – $O(k)$**
 - Расширяет список, добавляя **все** элементы из переданной коллекции.
 - `nums = [1, 2]; nums.extend([3, 4]) # -> [1, 2, 3, 4]`
- **`.insert(index, item)` – $O(n)$**
 - Вставляет элемент на заданную позицию, сдвигая остальные.
 - `nums = [1, 3]; nums.insert(1, 2) # -> [1, 2, 3]`

Удаление элементов

- **`.remove(value)` – $O(n)$**
 - Удаляет первое вхождение элемента по его значению.
 - `nums = ['a', 'b', 'c', 'b']; nums.remove('b') # -> ['a', 'c', 'b']`
 - Вызывает `ValueError`, если элемента нет.
- **`.pop(index=-1)` – $O(1)$ для конца, $O(n)$ для остальных**
 - Удаляет элемент по индексу и возвращает его.
 - `nums = [1, 2, 3]; last = nums.pop() # -> last=3, nums=[1, 2]`
- **Оператор `del`**
 - Удаляет по индексу или срезу. Ничего не возвращает.
 - `del nums[0]`
 - `del nums[1:3]`

Кортежи. `tuple`

- **упорядоченная, неизменяемая (`immutable`)** коллекция объектов произвольного типа.
- **Ключевые свойства:**
 - **Упорядоченный (`Ordered`):** Как и списки, сохраняют порядок добавления элементов.
 - **Неизменяемый (`Immutable`):** Главное отличие. После создания кортеж **нельзя** изменить.
 - **Гетерогенный (`Heterogeneous`):** Также может хранить объекты разных типов.

Кортежи. tuple

- литерал `()`

```
empty_tuple = ()
point_3d = (10, 20, 5)
person_data = ('Alice', 30, 'NSU')

print(empty_tuple, type(empty_tuple))
print(point_3d, type(point_3d))
print(person_data, type(person_data))
```

Output:

```
() <class 'tuple'>
(10, 20, 5) <class 'tuple'>
('Alice', 30, 'NSU') <class 'tuple'>
```

Кортежи. tuple

- конструктор `tuple()`

```
list_example = [1, 2, 3]
tuple_from_list = tuple(list_example)

print(tuple_from_list, type(tuple_from_list))
```

Output:

```
(1, 2, 3) <class 'tuple'>
```

Кортеж из одного элемента

```
not_a_tuple = (1)  
type(not_a_tuple)
```

```
is_a_tuple = (1,)  
type(is_a_tuple)
```

Output:

```
int  
tuple
```

Операции с кортежами

- **Можно:**

- **Индексация и срезы:** Работают точно так же, как в списках.

- `point_3d = (10, 20, 5)`
 - `print(point_3d[0])` # -> 10
 - `print(point_3d[1:])` # -> (20, 5)

- **Проверка на вхождение (in):**

- `print(20 in point_3d)` # -> True

- **Конкатенация (+) и повторение (*):** Создают новые кортежи.

- `new_tuple = (1, 2) + (3, 4)` # -> (1, 2, 3, 4)

- **Нельзя:**

- **Любые попытки изменения:**

- `# point_3d[0] = 15` # -> `TypeError: 'tuple' object does not support item assignment`
 - `# point_3d.append(3)` # -> `AttributeError: 'tuple' object has no attribute 'append'`
 - `# del point_3d[0]` # -> `TypeError: 'tuple' object doesn't support item deletion`

Зачем нужны кортежи?

- Защита данных от изменений (целостность данных)
- Ключи в словарях и элементы множеств

Распаковка кортежей

- **Базовая распаковка:**

- `point = (10, 20)`
- `x, y = point` *# x становится 10, y становится 20*

- **Обмен значениями**

- `a, b = 1, 2`
- `a, b = b, a` *# a становится 2, b становится 1*

- **Возврат нескольких значений из функции:**

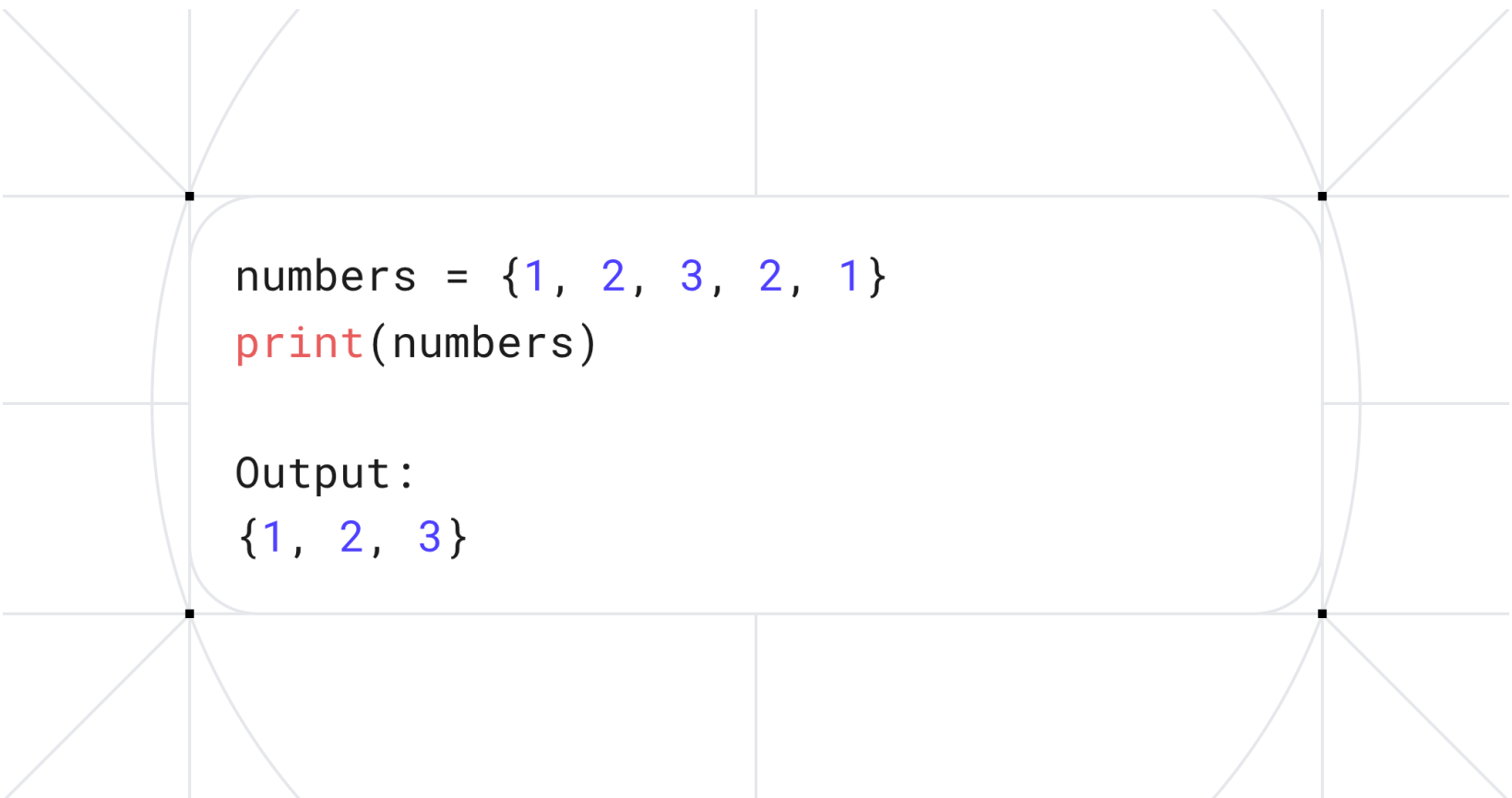
- `def get_min_max(numbers):`
- `return min(numbers), max(numbers)` *# возвращает кортеж*
- `my_nums = [1, 5, -3, 10]`
- `min_val, max_val = get_min_max(my_nums)`

Множества. `set`

- **Неупорядоченные изменяемые коллекции уникальных неизменяемых элементов.**
- **Ключевые свойства:**
 - **Неупорядоченный (Unordered):** Элементы не имеют определенного порядка или индекса.
 - **Изменяемый (Mutable):** Можно добавлять и удалять элементы.
 - **Уникальный (Unique):** Множество не может содержать два одинаковых элемента.
 - **Хэшируемые элементы:** Как и ключи словаря, элементы множества должны быть неизменяемыми.

Множества. `set`

- литерал `{}` (для непустых множеств)



```
numbers = {1, 2, 3, 2, 1}  
print(numbers)
```

```
Output:  
{1, 2, 3}
```

Множества. `set`

- конструктор `set()`

```
my_list = [1, 'a', 2, 'a', 3, 1]
unique_elements = set(my_list)
print(unique_elements)
```

Output:

```
{1, 2, 3, 'a'}
```

Пустое множество

```
a = {}  
b = set()  
print('a', type(a))  
print('b', type(b))
```

Output:

```
a <class 'dict'>  
b <class 'set'>
```

ОСНОВНЫЕ МЕТОДЫ

- **.add(item)**: Добавляет один элемент. Если он уже есть, ничего не происходит.
 - `s = {1, 2}`
 - `s.add(3) # -> {1, 2, 3}`
 - `s.add(2) # -> {1, 2, 3}`
- **.remove(item)**: Удаляет элемент. Вызывает ошибку **KeyError**, если элемента нет.
 - `s.remove(3) # -> {1, 2}`
 - `# s.remove(99) # -> KeyError`
- **.discard(item)**: Удаляет элемент. Не вызывает ошибку, если элемента нет.
 - `s.discard(2) # -> {1}`
 - `s.discard(99) # -> {1} (ничего не произошло)`

Математические операции

- **Объединение (Union):** Все элементы из обоих множеств.
 - Оператор: `|`
 - Метод: `.union()`
 - `s1 = {1, 2, 3}; s2 = {3, 4, 5}`
 - `s1 | s2 -> {1, 2, 3, 4, 5}`
- **Пересечение (Intersection):** Только общие для обоих множеств элементы.
 - Оператор: `&`
 - Метод: `.intersection()`
 - `s1 & s2 -> {3}`

Математические операции

- **Разность (Difference):** Элементы, которые есть в первом множестве, но нет во втором.
 - Оператор: $-$
 - Метод: `.difference()`
 - `s1 = {1, 2, 3}; s2 = {3, 4, 5}`
 - `s1 - s2 -> {1, 2}`
 - `s2 - s1 -> {4, 5}`
- **Симметрическая разность (Symmetric Difference):** Элементы, которые есть только в одном из множеств (но не в обоих).
 - Оператор: $\wedge$
 - Метод: `.symmetric_difference()`
 - `s1 ^ s2 -> {1, 2, 4, 5}`

Словари. `dict`

- неупорядоченные (исторически) изменяемые отображения **ключ-значение**
- Ключевые свойства:
 - **Отображение (Mapping)**: Хранит пары **ключ: значение**.
 - **Изменяемый (Mutable)**: Можно добавлять, изменять и удалять пары.
 - **Ключи уникальны и хэшируемы**: Не может быть двух одинаковых ключей; ключи должны быть неизменяемыми типами.
 - **Значения произвольны**: Значения могут быть любого типа, включая дубликаты.
 - **Упорядочены (с Python 3.7)**: Современные словари помнят порядок вставки, но на это не стоит закладывать в архитектуру.

Словари. dict

- литерал `{}`

```
empty_dict = {}
```

```
student = {  
    'name': 'Alice',  
    'age': 21,  
    'courses': ['MIT-6.006', 'MIT-8.01']  
}
```

```
print(empty_dict)
```

```
print(student)
```

Output:

```
{}
```

```
{'name': 'Alice', 'age': 21, 'courses': ['MIT-6.006', 'MIT-8.01']}
```

Словари. dict

- конструктор `dict()`

```
person = dict(name='Bob', age=42)
```

```
pairs = [('city', 'Cambridge'), ('state', 'MA')]  
location = dict(pairs)
```

```
print(person)  
print(location)
```

Output:

```
{'name': 'Bob', 'age': 42}  
{'city': 'Cambridge', 'state': 'MA'}
```

Доступ к значениям

1. Доступ по ключу []:

- Прямой, быстрый, идиоматичный.
- Вызывает `KeyError`, если ключа нет.
- `student = {'name': 'Alice', 'age': 21}`
- `print(student['name']) # -> 'Alice'`
- `# print(student['major']) # -> KeyError: 'major'`

2. Метод `.get(key, default=None)`:

- "Безопасный" доступ.
- Не вызывает ошибку. Возвращает `None` (или другое значение по умолчанию), если ключа нет.
- `major = student.get('major') # -> major = None`
- `major_default = student.get('major', 'Undeclared') # -> major_default = 'Undeclared'`

Модификация словарей

1. Добавление новой пары:

- `student = {'name': 'Alice'}`
- `student['age'] = 21` # Ключа 'age' не было, он будет создан
- `# student -> {'name': 'Alice', 'age': 21}`

2. Обновление существующего значения:

- `student['name'] = 'Alicia'` # Ключ 'name' уже был, его значение будет перезаписано
- `# student -> {'name': 'Alicia', 'age': 21}`

3. Метод `.update()` для слияния:

- `contact_info = {'email': 'alicia@mit.edu', 'phone': '555-1234'}`
- `student.update(contact_info)` # Добавляет/обновляет все пары из другого словаря

Удаление элементов

1. `del my_dict[key]:`

- Простое удаление по ключу. Вызывает `KeyError`, если ключа нет.
- `student = {'name': 'Alice', 'age': 21}`
- `del student['age']`
- `# student -> {'name': 'Alice'}`

2. `.pop(key, default):`

- Удаляет пару по ключу и **возвращает значение**. Безопасен: можно передать `default` значение, чтобы избежать `KeyError`.
- `student = {'name': 'Alice', 'age': 21}`
- `age = student.pop('age') # age -> 21`
- `major = student.pop('major', 'N/A') # major -> 'N/A', ошибки нет`

3. `.popitem():`

- Удаляет и возвращает **последнюю** добавленную пару (ключ, значение). Вызывает `KeyError`, если словарь пуст.

Итерация по словарю

1. Итерация по ключам (поведение по умолчанию):

```
student = {'name': 'Alice', 'age': 21}  
for k in student:  
    print(k)
```

Output:
name
age

Итерация по словарю

2. Итерация по значениям:

```
student = {'name': 'Alice', 'age': 21}
for v in student.values():
    print(v)
```

Output:

Alice

21

Итерация по словарю

3. Итерация по парам (ключ, значение)

```
for key, value in student.items():  
    print(f"{key}: {value}")
```

Output:
name: Alice
age: 21

Проверка на вхождение: `in`

- оператор `in` по умолчанию проверяет наличие **ключа**. Эта операция быстрая, $O(1)$.

```
student = {'name': 'Alice', 'age': 21}
```

```
print('name' in student)    # -> True (быстро,  $O(1)$ )
```

```
print('major' in student)   # -> False (быстро,  $O(1)$ )
```

Output:

```
True
```

```
False
```

Проверка на вхождение: `in`

- для проверки наличия значения нужно явно использовать `.values()`. Эта операция медленная, $O(n)$.

```
print('Alice' in student.values()) # -> True (медленно,  $O(n)$ )
```

Output:

```
True
```

Полезные материалы

- <https://docs.python.org/3/tutorial/datastructures.html>
- <https://realpython.com/python-lists-tuples>